Bounded Autonomy Through Environmental Composition: An Experimental Study of the GRAIL Runtime

Mike Amundsen
Independent Researcher
mca@mamund.com
ORCID: 0000-0002-9681-8338

Publication candidate v1.8 — 28 October 2026

Abstract

This study investigates whether explicitly composed, bounded environments can support goal-directed autonomous behavior without a general-purpose planning agent. The GRAIL runtime chooses unresolved conditions and affordances, invokes externally implemented capabilities, and updates an internal responsibility-oriented worldstate from reported outcomes. The analysis examines runtime source and a 37-suite regression record alongside eleven recorded scenario executions spanning six experiments: fallback, nested pursuits, three purchasing decisions, a dependency cycle with control, a false-success trust-boundary test with control, and an action-scoped failure-memory test with control. The positive cases demonstrate goal pursuit, alternative selection, recursive delegation, and heterogeneous bindings. Boundary cases show a cyclic pursuit failing to terminate within a three-second observation window, internal goal attainment without a corresponding external effect, and affordance-wide exclusion after a recorded failure. The paper proposes a preliminary operational semantics that distinguishes internal state from external domain truth and limits claims about termination, correctness, and generality. These observations support a bounded account of autonomy through environmental composition, not a universal correctness or performance guarantee.

1 Introduction

A system may act toward a goal without generating a global plan. In GRAIL, the environment defines conditions to satisfy, affordances that may produce them, and the capabilities that implement those affordances. The research question is: To what extent can autonomous behavior arise from environmental constraints and local decision-making without requiring a general-purpose planning or reasoning agent?

The study examines the narrower empirical proposition that a bounded, explicitly composed environment can support demonstrable goal pursuit, alternative selection after failure, and recursive delegation. These examples do not establish universal autonomy.

1.1 Research contributions and scope

This paper makes three contributions. First, it articulates *bounded operational autonomy through environmental composition* as a specific division of responsibilities: a world declares possibilities, prerequisites, and effects; a scenario supplies intent; and a coordinator chooses an unmet need and a means of addressing it, while capabilities implement domain operations. The claim concerns

delegated decision authority within a finite authored choice set, not general intelligence or a novel invention of precondition/effect representations.

Second, six experiments provide executable feasibility evidence. E1 demonstrates local alternative selection after failure; E2 demonstrates one form of recursive world composition and distinguishes successful responsibility completion from favorable domain outcomes; E3 demonstrates the same purchasing goal under approval, rejection, and deferral using heterogeneous bindings. These cases show that an LLM is not required in the coordination mechanism, although particular capabilities may use an LLM or human judgment.

Third, three experiments characterize boundaries of the architecture. E4 records a cyclic dependency that exceeds an external timeout, limiting any general termination claim. E5 shows internal goal attainment without a corresponding external effect when a capability falsely reports success. E6 shows affordance-wide failure exclusion across declared effects in the tested pursuit. These negative and controlled cases qualify the positive demonstrations rather than serving as implementation defects to conceal.

The study does *not* claim that the constituent mechanisms are new, that GRAIL outperforms workflow engines or planners, or that its monetary or engineering costs are lower. Architectural economy is a plausible motivation: coordination can avoid mandatory language-model inference, but environment authoring and capability development remain costs to be measured. The contribution is therefore an *empirically qualified architectural account*, not a comparative performance or cost-effectiveness result.

2 Foundations and Related Work

The idea of bounded autonomy has several precedents. Simon's bounded rationality distinguishes decision-making under constraints from unconstrained optimization [1]; his account of complex systems also motivates modular allocation of responsibilities [2]. Gibson's ecological affordances concern opportunities for action, while Norman emphasizes how designed environments communicate those opportunities [4, 5]. GRAIL borrows the term *affordance* for an explicitly authored software declaration, not a possibility perceived directly in a physical environment.

The distinction between a bounded decision space and a bounded decision policy matters. The environment author determines which needs and actions can be considered, while the selector determines which of the eligible choices is taken. This allocation resembles Simon's emphasis on decisions made with limited alternatives, but it does not claim that GRAIL's randomized selectors implement satisficing. Similarly, an authored affordance expresses a proposed software possibility rather than an ecological property discovered through perception. These analogies motivate the design without proving its novelty.

Classical planning already represents actions through prerequisites and effects [8, 9]. GRAIL does not invent this representation or compute a global plan; it resolves dependencies as a pursuit unfolds. Workflow systems likewise support branching and alternative execution paths, including declarative forms that cannot be characterized as fixed scripts [10]. The relevant comparison is therefore one of *allocation of control*, not a claim that existing systems lack alternatives. A path-prescriptive workflow fixes more of the invocation order in advance; a declarative workflow may leave comparable flexibility to execution time. Classical planners can also search over action alternatives. GRAIL's distinction lies in the particular combination of localized need-and-means selection, executable bindings, reported effects, and recorded pursuit observations, rather than in any single new planning primitive.

Situated-action research questions the primacy of complete advance plans [7]. Maes explores

goal-directed local action arbitration, Brooks challenges the necessity of centralized comprehensive representations, and behavior trees provide modular reactive control [3, 6, 11]. GRAIL differs in maintaining an explicit symbolic worldstate and choosing among declared producers of unmet conditions. It is neither representation-free nor equivalent to Maes's activation mechanism. Unlike Brooks' subsumption architecture, GRAIL explicitly represents symbolic conditions and effects. Unlike a behavior-tree control hierarchy, its pursuit does not necessarily follow a previously assembled control tree. These are differences in organization, not evidence that one architecture is superior. Belief–desire–intention architectures offer another comparison, although GRAIL does not implement a general BDI deliberation cycle [12].

For this study, *bounded operational autonomy* means delegated authority to select intermediate needs and available means, react to invocation feedback, and pursue an authored goal without a prescribed complete invocation sequence. It does not imply optimal choice, new capability invention, or verified external correctness. The contribution examined here is the combination of this allocation of authority with executable capability bindings and inspectable observations.

3 Environmental Composition

Environmental composition determines what a pursuit may attempt before the runtime chooses how to proceed. It supplies a bounded space of responsibilities and implementations, while leaving the sequence of eligible actions to local decisions. This distinction connects the intellectual precedents to the operational rules developed later in the paper.

3.1 From prescribed paths to authored possibility spaces

Environmental composition shifts part of the engineering task from specifying a sequence of operations to specifying the circumstances under which operations can become relevant. A GRAIL *world* declares conditions, affordances, prerequisites, effects, and bindings to operational capabilities. A *scenario* supplies a goal, initial worldstate, and inputs; *observations* preserve a record of attempted and completed activity. These three artifacts separate what may happen, what is sought, and what was reported to have happened. The distinction does not eliminate programming: authors must still design the world and implement capabilities. It instead changes what is fixed before execution. A workflow may prescribe a path, whereas a composed world may admit several paths that become distinguishable only during a pursuit. This is a contrast with path-prescriptive workflows, not with every workflow technology: declarative workflow systems and planning formalisms also allow runtime choice.

The value of composition therefore cannot rest on the novelty of preconditions and effects alone. STRIPS already represented actions through conditions and consequences [8]. GRAIL provides a concrete coordination architecture in which those declarations are paired with executable, independently implemented capabilities, two localized selection decisions, and inspectable pursuit observations. The claim examined here is modest: that this arrangement is sufficient to produce useful goal-directed behavior in several bounded software environments without requiring a general-purpose planner in the coordination runtime.

3.2 Possibility, eligibility, and reported consequence

Three elements define the immediate decision space. *Possibility* concerns the affordances the author makes available: the runtime cannot select an action that the world does not declare. *Eligibility* concerns prerequisites and available inputs: a declared action may exist but not yet be invocable.

Consequence concerns effects credited to a successful invocation: these change GRAIL's internal account of which responsibilities have been fulfilled. These are separable obligations. An affordance may be available in the registry but blocked by an unmet prerequisite; a capability may return an unfavorable business decision while successfully fulfilling its declared responsibility; and a capability may report success without producing a corresponding external change.

This last case is not a minor implementation detail. The internal state records what the runtime has reason to credit based on declared effects and reported invocation results. It does not constitute an independently verified description of the external domain. The environment's composition therefore establishes a boundary for action and attribution, not a universal guarantee about external truth. The operational model specifies this reporting rule, and E5 tests its consequence.

3.3 Two local decisions: needs and means

GRAIL separates selection of an unmet condition—a *need*—from selection of an affordance capable of satisfying it—a *means*. This separation matters because a pursuit need not commit in advance to one complete invocation sequence. Different unmet conditions may be selected, and a selected condition may have multiple candidate producers. The runtime can also use feedback from a failed invocation to exclude a previously attempted affordance and choose an alternative. The experimental evaluation examines both fallback and the scope of failure-based exclusion.

This constitutes *bounded operational autonomy*: authority to choose intermediate needs and means, and to respond to execution feedback, within an authored environment. Autonomy here refers to consequential delegated choice, not to the intelligence, optimality, or novelty of the selection policy. The inspected implementation uses randomized choice among eligible candidates with failure-based filtering; it does not implement a general-purpose planning algorithm or demonstrate adaptive learning. A choice can be operationally meaningful even when the selection rule is simple. Conversely, meaningful choice does not imply that the rule is efficient or that it will terminate on every world. Cyclic prerequisites remain a possible structural limitation, as the later boundary experiment demonstrates.

3.4 Responsibilities, outcomes, and composition cost

A capability is responsible for domain-specific work, including policy checks, external protocols, credentials, and human decisions. The runtime is responsible for coordinating the occurrence of declared responsibilities. A capability can therefore complete its assigned check successfully while returning a domain outcome such as `REJECTED` or `DEFERRED`. This separation permits heterogeneous capabilities and nested worlds to participate in the same pursuit without teaching the coordinator each domain's decision vocabulary.

The division is a deliberate trust boundary. The runtime can credit a reported successful invocation with its declared effects, but cannot independently establish that the external domain changed as claimed. It also relocates engineering effort: authors must define coherent conditions, dependencies, alternatives, and interfaces. A small coordination runtime does not imply that environment composition or verification is inexpensive. The experiments establish feasibility in bounded worlds, not a comparative cost advantage.

4 Experimental Apparatus

The experiments depend on a clear separation between the environment that declares possible work, the coordinator that selects and resolves it, and the capabilities that perform it. This section

describes those components as implemented in the tested worlds, then identifies the observations and controls used to examine their behavior.

4.1 Runtime, world, and scenario

GRAIL serves as the experimental coordination apparatus rather than as a domain-specific application. A *world* specifies available affordances and their capability bindings; a *scenario* supplies a goal, initial worldstate, and inputs; and *observations* record the resulting experience of a pursuit. This separation permits the same authored possibilities to be exercised under different initial conditions or domain inputs. In the examined examples, the registry expresses dependencies and declared effects rather than a complete ordered workflow. The environment author nevertheless determines the limits of possible action by deciding which affordances exist and what their declarations mean.

A pursuit begins with a named goal and an internal Boolean worldstate representing credited responsibilities. The coordinator attempts to establish unmet conditions using the authored affordances. Each affordance associates a name with prerequisite conditions, one or more declared effects, and a binding to an operational capability. Capabilities implement domain behavior outside the coordination mechanism. The apparatus thus distinguishes the *declaration* that a responsibility can be fulfilled from the *implementation* that attempts to fulfill it. The registry is not an executable proof of its own declarations.

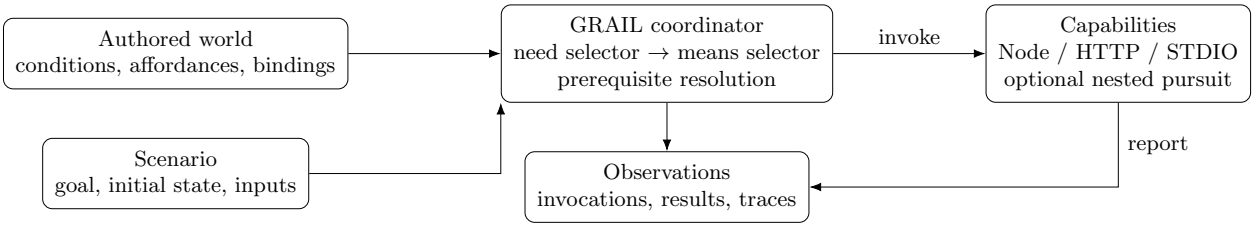


Figure 1: Responsibility boundaries in the experimental apparatus. Capability reports update internal credited worldstate; arrows do not imply independent verification of external effects.

4.2 Selection and execution

The inspected runtime exposes two distinct selection points. A need selector chooses an unmet condition to pursue; a means selector chooses an affordance that declares an effect capable of satisfying that condition. In the inspected implementation, both use randomized selection among applicable candidates, with exclusion of some previously failed affordances based on recorded observations. These policies are sufficient to produce different execution paths without a preauthored sequence, but the present study does not claim that they optimize path length, cost, or reliability. The runtime resolves prerequisites before invoking an affordance, records the invocation result, and credits declared effects after reported success. When an invocation fails, an alternative authored affordance may remain available. The execution trace exposes selected conditions, selected affordances, and stack operations; invocation observations capture operational results. Neither record should be mistaken for an independent audit of external state.

4.3 Capability boundary and composition

The studied worlds exercise Node module, HTTP, and STDIO bindings, including interactive STDIO for a human purchasing decision. The nested-onboarding world additionally uses an API helper to

invoke a child GRAIL pursuit from a parent capability. This is composition through the capability boundary: the parent coordinates a responsibility while the child independently pursues its own goal. The parent receives the child's reported result without needing to implement the child's verification procedure. The same division allows capabilities to return business outcomes such as `REJECTED`, `DENIED`, or `DEFERRED` while reporting successful completion of the responsibility assigned to them. It also creates a trust boundary: internal worldstate reflects reported completion, not necessarily independently verified external consequences.

4.4 Instrumentation and experimental controls

The archived evidence consists of scenario configuration files, capability source, complete JSON pursuit outputs, invocation observations, and decision/stack traces. Where relevant, the archives also include persisted domain records and process-harness reports. The boundary experiments add explicit controls. E4 changes an initial prerequisite from satisfied to unsatisfied and imposes an external process timeout; E5 changes whether a record is actually written while retaining a successful capability report; E6 controls selector choices to expose failure exclusion across multiple declared effects. These controls isolate particular architectural behaviors rather than provide statistical samples. The experimental method below specifies the resulting cases and consistency checks.

5 Operational Model

The preceding sections describe GRAIL in terms of its architecture, responsibilities, and execution environment. This section offers a more precise account of how those elements interact during a pursuit. The purpose of the operational model is not to prescribe an implementation or prove that every pursuit will terminate successfully. Instead, it identifies the information GRAIL maintains, the decisions it makes, and the conditions under which execution can proceed, fail, or stop.

The model uses mathematical notation to express these relationships and describe how a pursuit evolves during execution. The expressions formalize the concepts introduced earlier, with particular attention to the distinction between GRAIL's internal observations and the external outcomes that remain the responsibility of capabilities.

5.1 Worlds, pursuits, and the trust boundary

A composed world is modeled as $W = (C, A, \text{pre}, \text{eff}, \text{bind})$, where C is a finite set of named Boolean responsibility conditions, A is a finite set of authored affordances, $\text{pre}(a) \subseteq C$ gives the prerequisites of action a, $\text{eff}(a) \subseteq C$ gives its declared effects, and $\text{bind}(a)$ identifies its operational capability. A scenario supplies $g \in C$, an initial internal worldstate $s_0 : C \to \{0, 1\}$, and inputs i_0. A condition describes whether GRAIL has credited a responsibility as completed; it need not describe a favorable business outcome.

A pursuit is represented schematically by

$$q = \langle g, s, \kappa, I, O, T, F, m \rangle, \tag{1}$$

where κ is the obligation stack, I the current input/output environment, O the invocation observations, T the decision/stack trace, F the set of affordance actions excluded by applicable failure observations, and m a control mode. This is an *analytical abstraction*, not a claim that the implementation stores one object with precisely these fields. In particular, F summarizes information drawn from observations rather than requiring a dedicated runtime set.

To expose the trust boundary, let $d \in D$ denote external domain state, which the coordinator does not generally inspect. For a condition c, $\mathrm{Ext}(c, d)$ denotes the external proposition an application might intend c to represent. No invariant $s(c) = 1 \Rightarrow \mathrm{Ext}(c, d)$ is assumed. The model therefore separates *internal responsibility credit* from *external domain truth*.

5.2 Two decision functions

Define the producer set of a condition by

$$\mathrm{Prod}(c) = \{a \in A \mid c \in \mathrm{eff}(a)\}. \tag{2}$$

For an action a being resolved, its unmet prerequisites are

$$U(a, s) = \{c \in \mathrm{pre}(a) \mid s(c) = 0\}. \tag{3}$$

The *need selector* chooses $\sigma_N(q, a) \in U(a, s)$ when that set is nonempty. The *means selector* chooses $\sigma_M(q, c) \in \mathrm{Prod}(c) \setminus F$, subject to the runtime's other eligibility and resolution checks. For the initial goal, $c = g$ acts as the unresolved need. The inspected implementation uses randomized choice among available candidates, not a globally optimizing or learned policy. Thus selection is consequential but not evidence of sophisticated deliberation.

A crucial distinction concerns *selection* versus *invocation*. A producer may be selected to address a need even though its prerequisites remain false. The resolver then seeks producers for those prerequisites; it invokes the chosen capability only once the declared prerequisites and required inputs have been resolved. This distinction allows local choices to assemble an execution path incrementally, but it also creates the possibility of unresolved cycles.

5.3 Abstract transition obligations

The notation $q \longrightarrow q'$ denotes a coordination step and $\mathrm{report}(a) \in \{\mathrm{SUCCESS}, \mathrm{FAIL}\}$ for the outcome reported through a capability binding. The rules below specify the *observable obligations* of the coordination mechanism; they intentionally abstract away individual JavaScript stack operations, input-resolution details, and optional compensation.

Goal recognition (G). If $s(g) = 1$, the pursuit can terminate with reached = true. This conclusion concerns s, not d.

Need selection (N). For a selected action a with $U(a, s) \neq \varnothing$, choose $c = \sigma_N(q, a)$. Record the selection and introduce an obligation to establish c before invoking a.

Means selection (M). To resolve an unmet condition c, choose an available producer $a \in \mathrm{Prod}(c) \setminus F$, record the choice, and push its resolution obligation. If no producer is available, a blocked or failed resolution path may result; the precise reported status depends on the concrete runtime path.

Invocation (I). Invoke $\mathrm{bind}(a)$ only when $\forall c \in \mathrm{pre}(a),\ s(c) = 1$ and required inputs can be resolved. This is prerequisite discipline relative to the *declared internal state*; it does not check $\mathrm{Ext}(c, d)$.

Reported success (S). After report(a) = SUCCESS, append an observation, incorporate reported outputs into I, and credit the declared effects:

$$s'(c) = \begin{cases} 1 & c \in \text{eff}(a), \\ s(c) & \text{otherwise.} \end{cases} \tag{4}$$

The capability may have changed external state d, but Equation 4 does not establish that it did so. The domain result may also be `REJECTED`, `DENIED`, or `DEFERRED` while the invocation report is `SUCCESS`.

Reported failure (F). After report(a) = FAIL, record the failure, do not credit eff(a), and permit selection of an alternative producer. In the tested implementation the effective exclusion is action-scoped: $F' = F \cup \{a\}$ for the applicable observation context, rather than excluding only the pair (c, a). Optional compensating recovery is a distinct mechanism and is not modeled as forward fallback.

Unresolved resolution (B). A lack of usable producers or unresolved required inputs may terminate a pursuit without reaching the goal. No rule assumes that *all* unresolvable dependency structures will be detected. In particular, finite C and A do not imply a finite sequence of resolver steps.

5.4 Worked execution: local choices and fallback

Experiment E1 provides an execution witness for the distinction between needs and means. The authored world admits more than one producer for email verification and customer onboarding. During the recorded pursuit, GRAIL selected `verifyEmailUnavailable`; that capability reported failure because its provider was unavailable. The runtime subsequently selected `verifyCustomerEmail`, which succeeded. A second failure of `onboardCustomerUnavailable` was followed by successful selection of `onboardCustomer`. The pursuit ultimately reached `customerOnboarded`. These names identify recorded affordance choices, not an assumed universal selection order.

At the means-selection point, the environment supplied the candidate set and the runtime selected one member. Failure feedback changed which means remained available. At need-selection points, the resolver determined which unmet prerequisite to address next. Neither the specific failed-first path nor the successful alternatives had to be written as one fixed execution script. Conversely, the runtime did not invent a new capability: all choices were drawn from authored possibilities. E1 therefore demonstrates *delegated choice over a bounded action set*, not optimal planning or adaptive learning. The particular order of candidate selection is a recorded outcome of the tested run, not a deterministic guarantee.

The same formal distinction clarifies E6. There, `sharedProducer` declared two effects. After a failure while pursuing one condition, the runtime did not reconsider that action for the other condition in the tested pursuit; the alternatives supplied by the world completed the goal. The relevant exclusion scope is the affordance action, not the individual effect.

5.5 Properties, counterexamples, and limits

The model yields several *candidate* properties but does not establish their universal validity. Internal goal soundness (P1) says that a reported reached goal corresponds to $s(g) = 1$. Prerequisite discipline (P2) says an invocation is preceded by internal satisfaction of declared prerequisites. Both have

implementation-level support, not machine-checked proofs. Failure exclusion (P3) is action-scoped in the inspected implementation and witnessed by E6.

Termination (P4) requires stronger assumptions. In E4, the dependency cycle pre(`establishA`) = {`conditionB`} and pre(`establishB`) = {`conditionA`}, with both conditions initially false, repeatedly generated resolution obligations without an observed invocation. The harness killed the worker after approximately three seconds; the acyclic control completed. This is evidence against an unconditional practical termination guarantee for the tested resolver, not a mathematical proof of infinite execution.

External soundness (P5) is not entailed by reported success: E5 reached `recordCreated` in both scenarios even though the external record was absent in the false-success case. Recursive composition (P6) is demonstrated by the parent–child pursuit in E2, but the model supplies no general correctness theorem for arbitrary nesting. These qualifications identify the boundary between a useful operational semantics and guarantees that would require stronger contracts, additional assumptions, or formal verification.

6 Method

The evaluation tests whether the proposed division of responsibilities appears in executable pursuits and where it breaks down. Rather than estimating population-level performance, the study examines specific cases, paired controls, and recorded traces. This section explains how the cases were selected, checked, and subsequently exercised again.

6.1 Study design and units of analysis

The evaluation uses a case-based design comprising six experiment families and eleven recorded scenario executions. The unit of analysis is a *pursuit*: a goal-directed execution under an authored registry, initial worldstate, and inputs. E1–E3 are positive feasibility demonstrations; E4–E6 use paired scenarios to isolate termination, reported-success, and failure-memory behavior. The selected cases are not a random sample, and a single run does not estimate a success probability or average runtime cost.

The source material comprises world configurations, capability implementations, JSON pursuit outputs, observations, selection/stack traces, and, where applicable, persisted domain records and process-harness reports. The analysis also examines the supplied runtime source and a user-run regression record reporting 37 passing test suites. These regression tests support implementation-level plausibility; they are not independent replication of the eleven world executions.

6.2 Procedure and evidence checks

For each archived execution, the analysis identifies the configured goal and initial state, reads the complete pursuit output, counts SUCCESS and FAIL observations, and checks the event sequence recorded by the trace. Where standalone observation files are supplied, they are compared with observations embedded in the pursuit output. The checks establish consistency among saved artifacts, not independent truth of every capability report. In E3, the persisted purchasing store provides an additional domain-level check; in E5, the presence or absence of a written record is the independently examined variable.

E4's harness runs the API pursuit in a child process with an external three-second wall-clock timeout; termination by the harness is not a GRAIL terminal result. E6 uses controlled selector choices to make the multi-effect failure path reproducible. E3 includes interactive human decisions;

Table 1: Experimental design and observed outcomes. A dash indicates no paired control.

ID	Goal	Scenario contrast or intervention	Recorded outcome
E1	`customerOnboarded`	Two unavailable providers; alternate affordances	Goal reached after two failures
E2	`onboardingProcessed`	Parent invokes child verification world	Both goals reached; domain DENIED
E3	`purchaseRecorded`	Human APPROVE / REJECT / DEFER	Goal reached in all three runs
E4	`conditionA`	Prerequisite seeded true / unsatisfied cycle	Control reached; cycle timed out
E5	`recordCreated`	External record written / omitted despite SUCCESS	Both internally reached; external record differs
E6	`workCompleted`	Shared producer succeeds / fails, alternatives available	Both reached; failed producer excluded

elapsed times are therefore descriptive, not controlled measures of coordinator performance. Other selectors may select randomly, so one trace is a witness to a possible path rather than a claim about the distribution of paths.

6.3 Reproduction protocol and artifact identification

The reproducible unit is a *scenario execution*, not an experiment name alone. A replicator should retain the archived input, registry, initial worldstate, capability sources, original pursuit JSON, and trace as a single case record before attempting a new run. First, inspect the case's `README.md`, `package.json`, and lockfile (if present); record the Node version, operating system, dependency resolution, and GRAIL package actually loaded. Second, install dependencies from the case's declared manifest where possible and invoke the case's documented `api-run.js` or equivalent harness from its world root. Third, preserve fresh results under new filenames, rather than overwriting the archived observations. Finally, compare the goal status, invocation outcomes, selected affordances, and relevant external artifacts against the archived witness; compare durations only descriptively.

The experiment packages are not uniform: E1 includes `failure-recovery/api-run.js` and `config/`; E2 supplies a parent `api-run.js` and a nested child-world directory; E3 uses interactive purchasing inputs and may require its local HTTP store to be started; E4 uses `api-worker.js` and a harness-enforced timeout. E5 and E6 require their paired control and intervention scenarios to be run separately. These are *artifact entry points*, not a claim that a single command or package version reproduces every case. Before public release, a companion repository should pin dependency resolutions, document case-specific commands and human responses, and provide checksums for all eleven archived witnesses. The existing archives permit inspection and partial re-execution, but the available host-environment metadata are insufficient to assert bit-for-bit replication.

6.4 Fresh behavioral reproduction checks

After the archival analysis, the six experiment families were exercised again using locally available runtime dependencies and the archived world packages. These checks test the expected *behavioral contrasts*, not exact trace replay or clean-room installation. E1 reached its goal with one failed affordance, whereas its archived witness contained two failures; randomized affordance selection can account for different paths. E2 reached both parent and child goals and again reported domain outcomes `REJECTED` and `DENIED`, but used a known compatible local 0.3.0-beta.1 runtime rather

than an identified historical E2 revision. E3 reproduced all three purchasing decisions, with the HTTP service running and interactive input supplied through a pseudo-terminal; the persisted processing statuses matched the archived cases.

The E4 control completed, whereas its cyclic case exceeded the external three-second observation window. A working copy of its harness required a larger output buffer to avoid an earlier buffer-exhaustion failure; the original archived evidence was not altered. E5 reproduced the contrast between internal success with and without an external record. E6's packaged control and failure-memory tests passed in an existing dependency environment. These results constitute successful *behavioral re-execution across six experiment families*; they do not imply that all eleven original paths, timings, and traces were identically reproduced. Fresh package installation, E2's historical source revision, and fully pinned host dependencies remain unresolved.

7 Experiments and Results

The results are organized around the behavior each experiment was designed to expose. Positive cases establish executable feasibility; controlled boundary cases reveal limits on termination, external correctness, and failure exclusion. The reported counts and durations describe individual recorded pursuits, not comparative performance measurements.

7.1 Overview

Table 2 reports descriptive observations for the eleven executions. The distinction between a reported GRAIL goal and an independently checked domain consequence is particularly important for E2, E3, and E5. Durations are not compared statistically.

Table 2: Recorded pursuit results. Duration is the recorded pursuit elapsed time in milliseconds, not CPU time or coordinator-only overhead. E3 includes interactive human response latency. E4 instead reports separate wall-clock harness times in the last column; its timed-out case has no completed pursuit trace. Trace counts represent captured selection and stack events.

Scenario	Success	Fail	Trace	Pursuit (ms)	Goal / harness
E1 fallback	7	2	34	7.54	reached
E2 nested	4	0	15	43.15	reached
E3 approve	5	0	19	7465.47	reached
E3 reject	5	0	19	5193.75	reached
E3 defer	5	0	19	3482.50	reached
E4 control	1	0	3	—	reached; 227 ms harness
E4 cycle	0	0	—	—	timeout; 3077 ms harness
E5 truthful	1	0	3	6.69	reached
E5 false	1	0	3	6.32	reached
E6 control	2	0	7	7.05	reached
E6 failure	3	1	15	12.29	reached

7.2 E1: Alternative affordance fallback

The `customerOnboarded` pursuit records nine invocations: seven SUCCESS and two FAIL. The failures correspond to unavailable email-verification and onboarding providers. Subsequent selections choose working alternatives, and the goal is reached. Its 34-event trace contains nine affordance

selections, seven condition selections, nine stack pushes, and nine stack pops. This is a concrete witness that feedback can change the means selected during one pursuit; it is not an estimate of how often fallback succeeds. It concerns forward alternative selection, not compensating reversal of completed external effects.

7.3 E2: Nested customer onboarding

The parent world reaches `onboardingProcessed` after four successful parent invocations, including a capability that calls a child GRAIL world through the API. The child reaches `customerVerified`, while its domain verification outcome is `REJECTED`; the parent subsequently reports activation `DENIED`. Thus two reported goal completions coexist with unfavorable business decisions. The parent trace has 15 events; contemporaneous child observations are embedded in the parent output, but a complete child decision trace is not preserved. The case demonstrates one parent–child composition, not arbitrary-depth correctness.

7.4 E3: Purchase management

The same `purchaseRecorded` goal is reached after each of three interactive review decisions, with five successful invocations and 19 trace events per run. The world coordinates HTTP, Node, and interactive STDIO capabilities. Table 3 distinguishes domain outcomes from responsibility completion. The stored purchase records agree with the corresponding decisions. The durations (approximately 3.5–7.5 seconds) are strongly affected by the human review step and should not be interpreted as measurements of runtime efficiency.

Table 3: Purchase-management decisions and recorded business consequences.

Review status	Processing status	GRAIL goal
`approved`	`purchase-order-created`	reached
`rejected`	`request-closed-rejected`	reached
`deferred`	`follow-up-pending`	reached

7.5 E4: Cyclic prerequisite dependency

The positive control seeds `conditionB=true`, allowing `establishA` to execute and reach `conditionA`. The control worker finishes in 227 ms and records one successful invocation and three trace events. In the paired case, both conditions begin false; establishing A requires B and establishing B requires A. The worker repeatedly attempts prerequisite resolution, with no capability invocation observed, and is killed after approximately 3,077 ms. This establishes failure to terminate *within the externally imposed observation window*. It does not prove mathematically infinite execution or identify every class of dependency graph that may be problematic.

7.6 E5: False reported success

Both scenarios invoke `recordCreation`, receive SUCCESS, and internally reach `recordCreated`. In the truthful control, an external domain record exists. In the deliberately false-success scenario, the capability omits the external write while still reporting SUCCESS; no external record exists. Each run records one successful invocation and three trace events. The controlled contrast demonstrates that internal credited effects do not entail corresponding external facts. It does not estimate the prevalence of untruthful or defective capabilities in production.

7.7 E6: Failure-memory scope

The shared producer declares effects on both `conditionA` and `conditionB`. In the control, it succeeds once and a completion affordance reaches `workCompleted`. In the failure case, the shared producer fails while resolving A, followed by `alternativeA`, `alternativeB`, and `completeWork`, all successful. When B is later needed, the selection trace excludes the previously failed shared producer; its invocation count remains one. This demonstrates affordance-action-scoped exclusion across declared effects *within the tested pursuit*. It does not establish the lifetime of failure history across independent pursuits.

7.8 Evidence-to-contribution mapping

The first contribution (architectural formulation) is grounded in the source-level separation of condition and affordance selection and witnessed by E1 and E6 selection records. The second (executable feasibility) is supported by E1's alternatives, E2's nested invocation, and E3's heterogeneous execution with distinct domain outcomes. The third (boundary characterization) rests on E4's timeout contrast, E5's external-state contrast, and E6's action-wide exclusion. None of these observations alone establishes global optimality, general termination, external correctness, scalability, or lower cost than a comparator.

8 Discussion

The observations answer a narrow feasibility question, but they also raise a broader architectural one: what kind of autonomy follows when a runtime chooses within an authored environment? This section interprets the findings in terms of decision authority, allocation of control, trust, and the costs that such a design may shift rather than eliminate.

8.1 When does dependency resolution constitute autonomy?

A central interpretive question is whether GRAIL's execution amounts to autonomy or merely the mechanical resolution of dependencies. The distinction cannot be settled by counting actions or by requiring a particular algorithmic sophistication. A deterministic or randomized policy can exercise delegated decision authority, even when the policy is simple. Conversely, a complicated program may still execute an entirely prescribed path. Accordingly, *bounded operational autonomy* lies in the allocation of decisions: the environment author specifies a goal-relevant set of possibilities and constraints but does not fix every intermediate need, selected producer, or recovery choice in advance. During a pursuit, the coordinator selects an unresolved condition and then an affordance that can address it. Its decisions are consequential when more than one eligible route exists or when invocation feedback changes the remaining choices.

This definition is intentionally narrower than claims of intelligence, creativity, or general agency. Selecting randomly from alternatives does not imply that the selected alternative is good, efficient, or even likely to succeed. The current selectors do not learn an optimal policy from experience. The important empirical point is that the runtime holds a bounded degree of authority over the invocation sequence. E1 illustrates this authority when a failed producer is followed by a viable alternative; E6 shows that a failure can constrain subsequent means selection even for a different effect. In contrast, when a world contains exactly one viable producer at each step, the execution may be observationally indistinguishable from an ordinary dependency-driven procedure. Autonomy is therefore a *capacity of the composed decision space and coordinator*, not a property that every execution must display equally strongly.

This distinction also suggests an evaluation strategy beyond the present study. Future experiments could vary the number of viable producers, the structure of prerequisites, and the selector policy while holding the underlying domain task constant. The resulting traces could quantify how often selections genuinely change the path, how frequently feedback alters eligibility, and whether more informed selectors improve completion rates or reduce unnecessary invocations. Such a study would evaluate the *quality* of autonomous choice, which the present feasibility experiments do not measure.

8.2 Environmental composition and the location of control

GRAIL's architectural choice is best understood as a relocation of control rather than an elimination of control flow. The environment declares affordances, their prerequisites and effects, and bindings to capabilities; the runtime interprets those declarations against a goal and evolving internal worldstate. The environment author thus fixes a *space of permissible behavior*, while the coordinator determines one realized path through that space. A path-prescriptive workflow instead fixes more of the execution structure before the run. This contrast is useful but not categorical: declarative workflow languages, planners, rule engines, and behavior trees also separate specifications from some runtime choices [10, 8, 11]. This comparison does not establish that GRAIL is uniquely capable of this separation.

What the experiments add is an account of the particular division of responsibilities implemented by GRAIL. In E1, alternatives are declared in the world and selected during pursuit rather than hard-coded as a domain-specific exception sequence in the coordinator. In E2, a parent capability invokes a child pursuit, allowing the parent to rely on a completed verification responsibility without reproducing the child's internal operations. In E3, HTTP requests, Node functions, and interactive STDIO participate in one pursuit while purchasing policy remains outside the runtime. These cases illustrate *compositional substitution*: the coordination mechanism can remain largely stable while the operational mechanism behind a responsibility changes. They do not, by themselves, prove that substitution is frictionless. Input/output mismatches, inconsistent effect declarations, and capability-specific assumptions may require substantial engineering work.

The distinction between occurrence and outcome is especially consequential. GRAIL credits a declared responsibility when an invocation reports success; the capability interprets domain results such as `REJECTED`, `DENIED`, or `DEFERRED`. In E2 and E3, unfavorable or nonfinal business decisions coexist with successful pursuit completion. This is not an assertion that all business outcomes are interchangeable. Rather, it identifies the semantic boundary: the runtime tracks whether a responsibility was fulfilled according to its contract, while the capability decides what fulfillment means in its domain. An environment that needs separate downstream behavior for different domain results must encode appropriate responsibilities or supply capabilities that perform the necessary domain-sensitive work.

8.3 The cost of boundedness and trust

Explicit bounds simplify some coordination decisions because the runtime need only consider authored conditions and affordances. They also create obligations for the environment author. Missing producers can make a goal unreachable; inaccurate prerequisites can permit unsafe invocation or block useful work; and incorrect effect declarations can misrepresent progress. E4 demonstrates that even a small finite world can contain a dependency cycle that defeats timely completion. The cyclic scenario exceeded an external three-second limit, while its control completed. This establishes a concrete termination limitation under the tested configuration, not a proof of infinite execution or

a complete classification of cyclic graphs.

E5 exposes a different limitation. Both the truthful and false-success scenarios reached the internal goal, but only the truthful capability created the external record. This contrast shows that a coordinator can enforce the *form* of a reported occurrence without independently establishing the truth of its external consequence. The result is compatible with the deliberate trust boundary, yet it also implies a verification responsibility outside the basic transition model. Applications with stronger assurance needs may require capability-level checks, postcondition validation, external reconciliation, or independent observation. Such mechanisms would change the effective contract and could add latency, complexity, and cost. They should not be attributed to the evaluated runtime without implementation evidence.

E6 reveals a third tradeoff: a failed multi-effect affordance was excluded when a different declared effect later required a producer. Action-scoped failure memory is a conservative way to avoid immediately repeating an unsuccessful action. It can also discard opportunities where the same capability would have succeeded under a different input or responsibility. The observation supports a specific policy characterization within the tested pursuit; it does not establish that action-scoped exclusion is universally preferable. More granular failure policies might improve opportunity preservation while introducing additional state and decision complexity.

8.4 Architectural economy and the role of language models

The architecture offers a plausible route to economy by avoiding mandatory general-purpose reasoning for routine coordination. The examined runtime can select unmet conditions, choose producers, invoke capabilities, and respond to failures without calling a language model. That is a feasibility observation about the coordinator, not an argument that language models have no useful role. A capability may employ an LLM for classification, extraction, or drafting; a human may supply judgment through an interactive binding; and conventional code may implement deterministic policy. The same coordinator can delegate to each without needing to reproduce its internal reasoning.

Any claim of cost-effectiveness, however, requires a defined comparison and a complete accounting. *Coordination cost* includes selector computation, runtime overhead, observations, retries, and any inference used by coordination. *Composition cost* includes modeling responsibilities, writing and maintaining capabilities, defining bindings, validating contracts, and investigating failures. *Assurance cost* includes testing, monitoring, and external verification of reported effects. Moving work from the coordinator into the environment may lower one category while increasing another. The present experiments report execution behavior and limited timings, but they do not measure these costs against a matched workflow engine, planner, or LLM-based agent.

A suitable comparative study would implement equivalent tasks under clearly specified success criteria, hold external services and input distributions constant, and measure both development effort and repeated operating expense. It should account for the quality of outcomes and the cost of recovering from failure, not merely count model calls or runtime milliseconds. The narrower conclusion available here is that *mandatory LLM inference is not a prerequisite for the demonstrated form of bounded goal-directed coordination.* Whether this yields lower total cost depends on the task, the available capabilities, the stability of the environment, and the required assurance level.

8.5 Implications for research and practice

The combined evidence suggests three implications. First, autonomy should be evaluated along at least two axes: *decision authority* (which choices the coordinator may make) and *decision quality* (how well its policy makes them). The present work primarily demonstrates the former. Second,

composition creates a practical separation between coordination and domain execution, but this separation relies on declarations and contracts whose accuracy is not guaranteed by the coordinator. Third, negative experiments are essential for understanding bounded systems: E4–E6 identify limitations that successful goal-reaching traces alone would not reveal.

9 Threats to Validity

The conclusions depend on how autonomy is defined, how the experimental cases were constructed, and what the recorded artifacts can establish. The following assessment separates limits of the measured concepts from limits of causal inference, generalization, and reproduction. These distinctions constrain the scope of the study's claims.

9.1 Construct validity

The study operationalizes *bounded autonomy* as delegated authority to select intermediate needs and means within a predefined environment. This definition is narrower than general intelligence, self-directed goal formation, or optimal planning. An observed change in action sequence is evidence of runtime selection authority; it does not by itself demonstrate sophisticated reasoning. Similarly, the Boolean worldstate measures credited responsibility completion, not necessarily the state of an external business system. E5 deliberately exposes this distinction: internal goal attainment occurs in both scenarios although only one creates the external record. Treating the internal flag as an independent measure of domain success would invalidate the interpretation.

9.2 Internal validity

The worlds and controls were constructed to exercise particular mechanisms rather than sampled from production workloads. Randomized selectors may yield different paths on repetition; E6 therefore fixes selector choices for a reproducible test of failure-memory scope. A single successful pursuit cannot estimate the probability of eventual success, and one selected path does not establish how often that path is taken. E4's three-second timeout demonstrates failure to terminate within the observation window, not infinite execution as a mathematical property. E3's human-review durations include human response time and cannot be attributed to runtime coordination overhead. Persistent observation histories may also affect later affordance eligibility unless isolated or reset; the available records do not support a general claim of history-independent behavior.

9.3 External validity

The experimental environments are deliberately small, authored, and bounded. They cover fallback, recursive delegation, heterogeneous bindings, and selected boundary cases, but not large-scale production integrations, adversarial services, long-running durable pursuits, or arbitrary dependency graphs. Their results establish feasibility and counterexamples in the tested configurations, not scalability, comparative reliability, or universal termination. In particular, a successful nested parent–child execution does not prove correctness of arbitrary recursive compositions. Likewise, a deliberately false capability report demonstrates the architectural trust boundary but is not evidence that deployed capabilities commonly misreport outcomes.

9.4 Reproducibility and evidential limits

Fresh behavioral re-execution covered all six experiment families, but did not establish clean-room or bit-for-bit reproducibility. Archived configurations, source, outputs, observations, and traces permit examination of what was recorded and support attempts at reproduction. They do not guarantee exact replay when selector randomness, interactive decisions, external HTTP state, or side effects differ. E2's separately saved child observations are not identical to the child observations embedded in the parent execution, and a complete child decision trace was not preserved for that run. Version provenance is mixed across the supplied packages, and exact host environment manifests are incomplete. The 37 passing regression suites are valuable corroboration of runtime behavior, but they are neither independent replications nor substitutes for the eleven scenario records. These limits motivate treating the results as a transparent case-based study rather than a controlled comparative benchmark.

10 Future Research

The present findings suggest a research agenda centered on the consequences of distributing decision authority and operational responsibility. The first priority is *termination and dependency analysis*. E4 identifies a cyclic dependency that the observed runtime did not resolve within the imposed limit. Future work should examine cycle detection, visited-state tracking, explicit resource bounds, and the conditions under which termination can be proved for restricted classes of worlds. Such work should distinguish a runtime's ability to report `BLOCKED` from its ability to recognize a dependency cycle before repeated resolution.

A second direction concerns *selection policies*. The two selectors form distinct intervention points: one prioritizes unmet needs and the other selects available means. Controlled comparisons could evaluate random choice, cost-aware heuristics, reliability-aware policies, and feedback-informed policies under fixed worlds and inputs. Measures should include goal attainment, invocation count, fallback frequency, time, and externally verified outcome quality. The distinction between decision authority and decision quality would then become experimentally measurable rather than merely conceptual.

Third, *capability contracts and external verification* deserve attention. E5 demonstrates that declared effects and reported success do not ensure external truth. Explicit input/output contracts, effect-specific evidence, idempotency declarations, and optional verification capabilities may strengthen confidence without requiring the coordinator to internalize domain logic. Any such mechanism must account for the costs of verification and for operations whose effects cannot be observed immediately.

Fourth, *failure-memory policy* merits systematic study. E6 demonstrates exclusion of a failed multi-effect affordance when another of its effects is later needed. More precise policies might distinguish an affordance's identity, its input context, its target condition, and transient versus persistent failure. Their benefits and risks should be evaluated against repeated side effects and unsafe retries rather than presumed from additional flexibility.

Finally, *replay, durable pursuits, and comparative architectural economy* remain open. Reproducible replay requires control or recording of random choices, human inputs, capability responses, and external side effects; traces alone are insufficient. Longer-lived and nested pursuits raise additional questions about state persistence and cross-world failure propagation. To test the proposed economic motivation, future comparisons should measure both coordination expense (including any inference calls) and composition expense (authoring, testing, maintenance, and verification). The present paper does not infer cost superiority from the absence of mandatory LLM inference.

11 Research artifact availability

The experimental worlds, original scenario observations, evidence register, and reproduction reports are intended for a companion research repository. At the time of this publication candidate, no immutable public release identifier or archival DOI has been assigned. Consequently, the artifact collection should not yet be described as independently retrievable from a permanent public record. The study reports successful behavioral re-execution across six experiment families, not bit-for-bit replay or clean-room reproduction. The historical local runtime revision for E2 remains unidentified. A public release should preserve the eleven original case records separately from fresh reproduction outputs and supply versioned installation and execution instructions.

12 Conclusion

The eleven recorded executions and subsequent behavioral checks support a bounded claim: an explicitly composed environment can enable a coordinator to pursue goals by selecting unmet conditions and authored affordances, responding to feedback, and delegating domain work without mandatory general-purpose planning or LLM inference. The feasibility cases demonstrate fallback, nested pursuit, and heterogeneous bindings; the boundary cases expose nontermination within a fixed observation window, the gap between reported and external success, and action-scoped failure exclusion. These findings characterize a division of authority and trust, not a guarantee of correctness, termination, optimality, or lower cost. The next research question is how different selectors, contracts, and verification policies change the quality and economics of this bounded autonomy.

References

[1] H. A. Simon, "A Behavioral Model of Rational Choice," *The Quarterly Journal of Economics*, 69(1), 99–118, 1955. DOI: 10.2307/1884852.

[2] H. A. Simon, "The Architecture of Complexity," *Proceedings of the American Philosophical Society*, 106(6), 467–482, 1962. Stable article record.

[3] P. Maes, "Situated Agents Can Have Goals," *Robotics and Autonomous Systems*, 6(1–2), 49–70, 1990. DOI: 10.1016/S0921-8890(05)80028-4.

[4] J. J. Gibson, *The Ecological Approach to Visual Perception*. Houghton Mifflin, 1979.

[5] D. A. Norman, *The Psychology of Everyday Things*. Basic Books, 1988.

[6] R. A. Brooks, "Intelligence without Representation," *Artificial Intelligence*, 47(1–3), 139–159, 1991. DOI: 10.1016/0004-3702(91)90053-M.

[7] L. A. Suchman, *Plans and Situated Actions: The Problem of Human-Machine Communication*. Cambridge University Press, 1987.

[8] R. E. Fikes and N. J. Nilsson, "STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving," *Artificial Intelligence*, 2(3–4), 189–208, 1971. DOI: 10.1016/0004-3702(71)90010-5.

[9] D. McDermott et al., *PDDL—The Planning Domain Definition Language*, AIPS-98 Planning Competition Committee, 1998. Yale Center for Computational Vision and Control, CVC TR-98-003/DCS TR-1165; report.

[10] W. M. P. van der Aalst, A. H. M. ter Hofstede, B. Kiepuszewski, and A. P. Barros, "Workflow Patterns," *Distributed and Parallel Databases*, 14(1), 5–51, 2003. DOI: 10.1023/A:1022883727209.

[11] M. Colledanchise and P. Ögren, *Behavior Trees in Robotics and AI: An Introduction*. CRC Press, 2018. Electronic-edition DOI: 10.1201/9780429489105.

[12] A. S. Rao and M. P. Georgeff, "BDI Agents: From Theory to Practice," *Proceedings of the First International Conference on Multi-Agent Systems*, 1995. pp. 312–319; original conference paper.